

Back To Practice

Q1

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 int main() {
5     int n, m;
6
7     // Input: Number of processes and resource types
8     printf("Enter number of processes: \n");
9     if (scanf("%d", &n) != 1) return 0;
10    printf("Enter number of resource types: \n");
11    if (scanf("%d", &m) != 1) return 0;
12
13    int alloc[n][m], max[n][m], avail[m], need[n][m];
14
15    // Input: Allocation Matrix
16    printf("Enter Allocation Matrix:\n");
17    for (int i = 0; i < n; i++) {
18        for (int j = 0; j < m; j++) {
19            scanf("%d", &alloc[i][j]);
20        }
21    }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Back To Practice

Q1

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

```
74 }
75
76 // If no process could be completed in this pass, we are stuck
77 if (!found) break;
78 }
79
80 // Final Output
81 if (count == n) {
82     printf("System is in a SAFE STATE.\n");
83     printf("Safe Sequence: ");
84     for (int i = 0; i < n; i++) {
85         printf("%d ", safeSeq[i]);
86     }
87     printf("\n");
88 } else {
89     printf("System is NOT in a safe state (Deadlock may occur).\n");
90 }
91
92 return 0;
93 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1